

Cahier des Charges - Application Météorologique Agricole pour Riziculteurs

1. Présentation du Projet

1.1 Contexte

Développement d'une solution technologique complète pour accompagner les riziculteurs dans leurs prises de décisions agricoles en s'appuyant sur des données météorologiques précises et des algorithmes de conseil agricole intelligent.

1.2 Objectifs

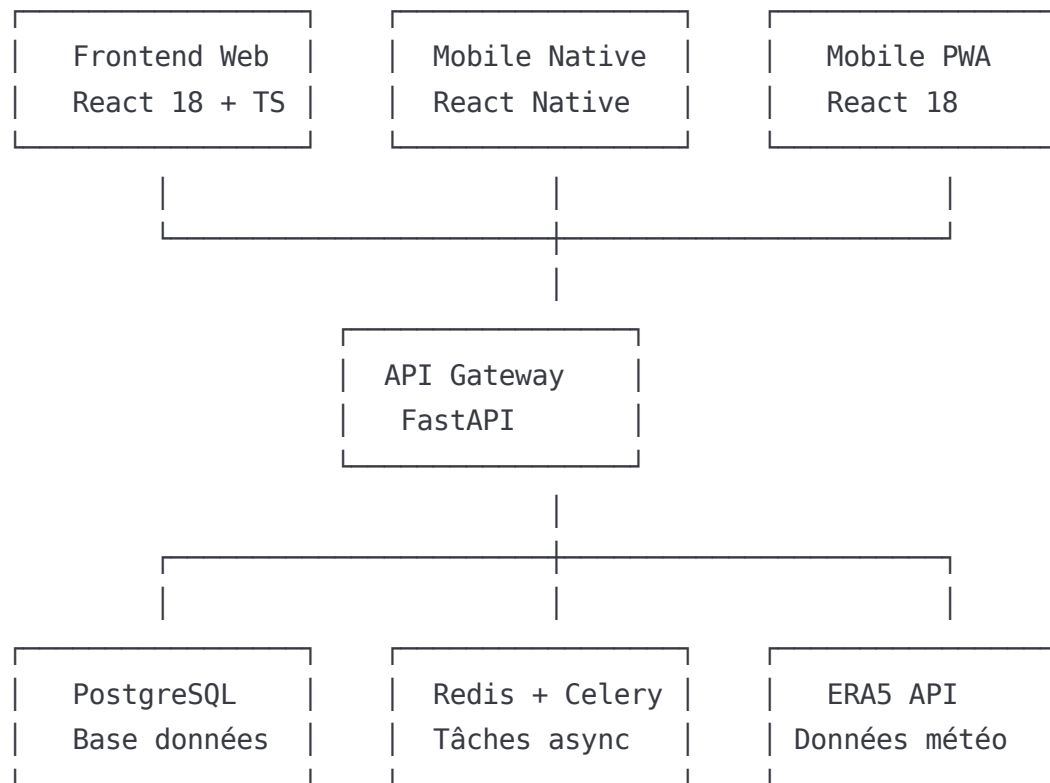
- Fournir des prévisions météorologiques précises et localisées
- Optimiser les décisions de semis et de gestion des cultures de riz
- Réduire les risques climatiques et améliorer les rendements
- Démocratiser l'accès aux données météorologiques pour les petits et moyens riziculteurs

1.3 Public Cible

- Riziculteurs individuels (petites et moyennes exploitations)
- Coopératives agricoles
- Conseillers agricoles et agronomes
- Organismes de développement rural

2. Architecture Technique

2.1 Vue d'Ensemble



2.2 Stack Technologique

Backend

- **Langage** : Python 3.11+
- **Framework** : FastAPI
- **Base de données** : PostgreSQL 15+
- **ORM** : SQLAlchemy 2.0
- **Cache** : Redis 7+
- **Tâches asynchrones** : Celery
- **Authentification** : JWT + OAuth2
- **Documentation** : Swagger/OpenAPI automatique

Frontend Web

- **Framework** : React 18
- **Langage** : TypeScript 5+
- **UI/UX** : Tailwind CSS 3+
- **État global** : Zustand ou Redux Toolkit
- **Graphiques** : Chart.js ou Recharts
- **Cartes** : Leaflet + OpenStreetMap
- **PWA** : Service Workers, Manifest

Mobile Native

- **Framework** : React Native 0.72+
- **Plateforme** : Expo SDK 49+
- **Navigation** : React Navigation 6
- **État** : Zustand
- **Cartes** : React Native Maps
- **Notifications** : Expo Notifications

3. Spécifications Fonctionnelles

3.1 Gestion des Utilisateurs

3.1.1 Inscription et Authentification

- **Inscription** par email/téléphone avec vérification
- **Connexion** multi-facteurs (SMS/Email)
- **Profils utilisateur** :
 - Riziculteur individuel
 - Coopérative
 - Conseiller agricole
- **Gestion des exploitations** multiples par utilisateur

3.1.2 Profil Agricole

- **Localisation** des parcelles (GPS, adresse)
- **Superficie** et caractéristiques des parcelles
- **Variétés de riz** cultivées
- **Calendrier cultural** personnalisé
- **Historique** des campagnes agricoles

3.2 Données Météorologiques

3.2.1 Sources de Données

- **ERA5 Reanalysis** (données historiques 1940-présent)
- **ERA5-Land** (résolution 9km, données terrestres)
- **Forecasts** ECMWF (prévisions 10 jours)
- **Données satellitaires** complémentaires

3.2.2 Paramètres Météorologiques

- **Température** (min, max, moyenne)
- **Précipitations** (cumul, intensité)
- **Humidité relative**
- **Vitesse et direction du vent**
- **Rayonnement solaire**
- **Évapotranspiration potentielle**
- **Indices agro-climatiques** spécialisés

3.2.3 Résolution et Couverture

- **Résolution spatiale** : 0.1° (~11km)
- **Résolution temporelle** : horaire, journalière, décadaire
- **Couverture** : Mondiale avec focus régional
- **Historique** : 40+ années de données

3.3 Conseils Agricoles Intelligents

3.3.1 Algorithmes de Décision

- **Modèles phénologiques** pour le riz
- **Calcul des sommes de températures**
- **Indices de stress hydrique**
- **Probabilités de risques climatiques**
- **Optimisation des fenêtres de semis**

3.3.2 Recommandations Personnalisées

- **Dates optimales de semis** selon variété et localisation
- **Gestion de l'irrigation** basée sur ETc
- **Alertes météorologiques** critiques
- **Conseils de protection** (gel, vent, pluie excessive)
- **Planification des opérations** culturales

3.3.3 Machine Learning

- **Apprentissage supervisé** sur données historiques
- **Modèles prédictifs** de rendement
- **Classification automatique** des risques
- **Amélioration continue** par feedback utilisateur

3.4 Interface Utilisateur

3.4.1 Dashboard Principal

- **Vue d'ensemble** de toutes les parcelles
- **Météo actuelle** et prévisions 7 jours
- **Alertes** et notifications prioritaires
- **Calendrier agricole** avec rappels
- **Graphiques** d'évolution des paramètres

3.4.2 Gestion des Parcelles

- **Cartographie interactive** avec géolocalisation
- **Fiches détaillées** par parcelle
- **Historique météorologique** et cultural
- **Comparaison** entre parcelles
- **Export** de données et rapports

3.4.3 Prévisions et Analyses

- **Cartes météorologiques** interactives
- **Graphiques temporels** multi-paramètres
- **Indices agro-climatiques** calculés
- **Scénarios** et projections
- **Recommandations** contextualisées

4. Spécifications Techniques Détaillées

4.1 Backend API (FastAPI)

4.1.1 Architecture

```
src/
├─ app/
│   ├── api/
│   │   ├── endpoints/
│   │   │   ├── auth.py
│   │   │   ├── users.py
│   │   │   ├── parcelles.py
│   │   │   ├── meteo.py
│   │   │   └─ conseils.py
│   │   └─ deps.py
│   ├── core/
│   │   ├── config.py
│   │   ├── security.py
│   │   └─ database.py
│   ├── models/
│   ├── schemas/
│   ├── services/
│   └─ tasks/
├─ tests/
└─ requirements.txt
```

4.1.2 Endpoints Principaux

- **Auth**: `/auth/login`, `/auth/register`, `/auth/refresh`
- **Users**: `/users/profile`, `/users/preferences`
- **Parcelles**: `/parcelles/`, `/parcelles/{id}/meteo`
- **Météo**: `/meteo/current`, `/meteo/forecast`, `/meteo/historical`
- **Conseils**: `/conseils/semis`, `/conseils/irrigation`

4.1.3 Base de Données PostgreSQL

sql

-- Schéma principal

CREATE SCHEMA agricole;

-- Tables principales

TABLE users (

id UUID PRIMARY KEY,
email VARCHAR UNIQUE,
phone VARCHAR,
profile_type ENUM,
created_at TIMESTAMP

);

TABLE parcelles (

id UUID PRIMARY KEY,
user_id UUID REFERENCES users(id),
nom VARCHAR,
latitude DECIMAL(10,8),
longitude DECIMAL(11,8),
superficie DECIMAL,
variete_riz VARCHAR,
created_at TIMESTAMP

);

TABLE donnees_meteo (

id BIGSERIAL PRIMARY KEY,
parcelle_id UUID REFERENCES parcelles(id),
date_heure TIMESTAMP,
temperature DECIMAL,
precipitation DECIMAL,
humidite DECIMAL,
vent_vitesse DECIMAL,
vent_direction INTEGER

);

```
TABLE conseils (  
  id UUID PRIMARY KEY,  
  parcelle_id UUID REFERENCES parcelles(id),  
  type_conseil VARCHAR,  
  contenu JSONB,  
  priorite INTEGER,  
  date_generation TIMESTAMP,  
  statut VARCHAR  
);
```

4.2 Frontend Web (React + TypeScript)

4.2.1 Structure du Projet

```
src/  
├─ components/  
│   ├─ common/  
│   ├─ charts/  
│   ├─ maps/  
│   └─ forms/  
├─ pages/  
│   ├─ Dashboard/  
│   ├─ Parcelles/  
│   ├─ Meteo/  
│   └─ Conseils/  
├─ hooks/  
├─ services/  
├─ stores/  
├─ types/  
└─ utils/
```

4.2.2 Composants Clés

- **WeatherChart** : Graphiques météorologiques interactifs
- **ParcelleMap** : Carte avec géolocalisation des parcelles
- **ConseilCard** : Affichage des recommandations
- **NotificationCenter** : Gestion des alertes
- **DataExport** : Export PDF/Excel des données

4.2.3 PWA Features

- **Service Worker** pour cache offline
- **Manifest** pour installation
- **Notifications Push** pour alertes
- **Synchronisation** background
- **Mode hors-ligne** limité

4.3 Mobile Native (React Native + Expo)

4.3.1 Fonctionnalités Spécifiques

- **Géolocalisation** automatique des parcelles
- **Photos** des cultures avec géomarquage
- **Notifications push** contextuelles
- **Mode offline** avec synchronisation
- **Capteurs** du téléphone (orientation, luminosité)

4.3.2 Compatibilité

- **Android** : Version 8.0+ (API 26+)
- **iOS** : Version 13.0+
- **Taille** : < 50MB installée
- **Performance** : 60fps sur appareils moyens

5. Intégration ERA5

5.1 API Climate Data Store (CDS)

- **Authentification** via clé API Copernicus
- **Requêtes** automatisées via cdsapi
- **Paramètres** : temperature, precipitation, wind, humidity
- **Format** : NetCDF4 puis conversion JSON/PostgreSQL

5.2 Pipeline de Données

python

Exemple de workflow

1. Requête CDS API pour zone géographique
2. Téléchargement fichiers NetCDF
3. Traitement et extraction des points de grille
4. Interpolation spatiale pour parcelles utilisateur
5. Calcul d'indices agro-climatiques
6. Stockage en base de données
7. Mise à jour cache Redis

5.3 Tâches Celery

- **Mise à jour quotidienne** des prévisions
- **Import historique** par chunks
- **Calcul d'indices** en arrière-plan
- **Nettoyage** des données anciennes
- **Backup** automatique

6. Algorithmes Agricoles

6.1 Modèles Phénologiques

6.1.1 Stades de Développement du Riz

python

Calcul des degrés-jours (GDD)

```
def calculate_gdd(temp_min, temp_max, base_temp=10, ceiling_temp=30):  
    temp_mean = (temp_min + temp_max) / 2  
    temp_mean = min(max(temp_mean, base_temp), ceiling_temp)  
    return max(0, temp_mean - base_temp)
```

Stades phénologiques

```
STAGES = {  
    'semis_levee': 150,      # GDD requis  
    'tallage': 400,  
    'montaison': 800,  
    'floraison': 1200,  
    'maturation': 1600  
}
```

6.1.2 Besoins en Eau

python

```
def calculate_etc(et0, kc_stage, growth_stage):  
    """  
    ET0: Évapotranspiration de référence  
    Kc: Coefficient cultural selon stade  
    """  
    kc_values = {  
        'initial': 1.05,  
        'development': 1.10,  
        'mid_season': 1.20,  
        'late_season': 0.90  
    }  
    return et0 * kc_values[growth_stage]
```

6.2 Indices de Risque

6.2.1 Stress Hydrique

- **Déficit hydrique** cumulé
- **Indice de satisfaction** des besoins
- **Probabilité de sécheresse**

6.2.2 Risques Climatiques

- **Gel** : température < 2°C au stade critique
- **Vent fort** : > 60 km/h pendant floraison
- **Pluies excessives** : > 50mm/jour

6.3 Optimisation des Semis

6.3.1 Algorithme Principal

python

```
def optimize_sowing_date(parcelle, variete, year):  
    """  
    Optimise la date de semis en fonction de :  
    - Données météo historiques  
    - Caractéristiques de la variété  
    - Contraintes locales  
    """  
    best_dates = []  
  
    for date in possible_dates:  
        # Simulation de la campagne  
        risk_score = calculate_risks(date, parcelle, variete)  
        yield_potential = estimate_yield(date, parcelle, variete)  
  
        score = yield_potential - risk_score  
        best_dates.append((date, score))  
  
    return sorted(best_dates, key=lambda x: x[1], reverse=True)
```

7. Sécurité et Performance

7.1 Sécurité

7.1.1 Authentification

- **JWT** avec refresh tokens
- **Rate limiting** sur API
- **Validation** stricte des inputs
- **Chiffrement** des données sensibles
- **CORS** configuré correctement

7.1.2 Protection des Données

- **RGD** compliance
- **Anonymisation** des données agricoles
- **Audit trail** des modifications
- **Backup** chiffrés automatiques

7.2 Performance

7.2.1 Optimisations Backend

- **Cache Redis** pour requêtes fréquentes
- **Indexation** PostgreSQL optimisée
- **Pagination** des résultats
- **Compression** des réponses API
- **Pool de connexions** DB

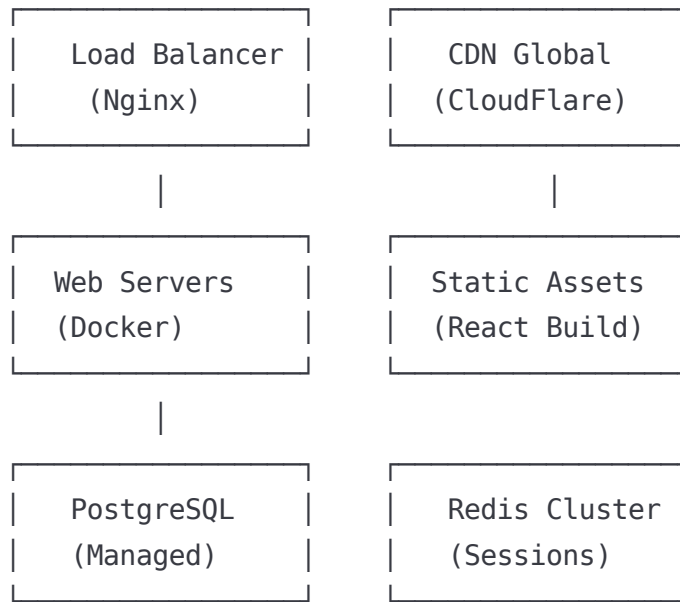
7.2.2 Optimisations Frontend

- **Code splitting** par routes
- **Lazy loading** des composants
- **Optimisation** des images
- **Service Workers** pour cache
- **Bundle** analysis et optimisation

8. Déploiement

8.1 Infrastructure

8.1.1 Architecture Cloud



8.1.2 Containerisation

dockerfile

Backend

FROM python:3.11-slim

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0"]

Frontend

FROM node:18-alpine

COPY package*.json ./

RUN npm ci --only=production

COPY . .

RUN npm run build

EXPOSE 3000

CMD ["npm", "start"]

8.2 CI/CD Pipeline

8.2.1 Workflow GitHub Actions

yaml

```
name: Deploy Production
on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Run Tests
        run: |
          python -m pytest
          npm test

  deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to Production
        run: |
          docker build -t app:latest .
          docker push registry/app:latest
          kubectl apply -f k8s/
```

9. Tests et Qualité

9.1 Stratégie de Tests

9.1.1 Backend

- **Tests unitaires** : pytest (>90% couverture)
- **Tests d'intégration** : API endpoints
- **Tests de performance** : locust
- **Tests sécurité** : bandit, safety

9.1.2 Frontend

- **Tests unitaires** : Jest + Testing Library
- **Tests composants** : Storybook
- **Tests E2E** : Playwright
- **Tests performance** : Lighthouse CI

9.2 Monitoring

9.2.1 Observabilité

- **Logs** : ELK Stack ou équivalent
- **Métriques** : Prometheus + Grafana
- **Tracing** : Jaeger
- **Alertes** : PagerDuty/Slack

9.2.2 KPIs

- **Performance** : temps de réponse API < 200ms
- **Disponibilité** : uptime > 99.9%
- **Erreurs** : taux d'erreur < 0.1%
- **Usage** : sessions actives, rétention

10. Planning de Développement

10.1 Phase 1 : Fondations (6 semaines)

- **Semaine 1-2** : Setup infrastructure et backend MVP
- **Semaine 3-4** : API authentification et gestion utilisateurs
- **Semaine 5-6** : Frontend de base et intégration ERA5

10.2 Phase 2 : Core Features (8 semaines)

- **Semaine 7-10** : Système de parcelles et données météo
- **Semaine 11-14** : Algorithmes agricoles de base

10.3 Phase 3 : Intelligence (6 semaines)

- **Semaine 15-18** : Conseils agricoles avancés
- **Semaine 19-20** : Machine Learning et optimisations

10.4 Phase 4 : Mobile & PWA (4 semaines)

- **Semaine 21-22** : Application React Native
- **Semaine 23-24** : PWA et fonctionnalités offline

10.5 Phase 5 : Production (4 semaines)

- **Semaine 25-26** : Tests, sécurité, performance
- **Semaine 27-28** : Déploiement et monitoring

11. Budget et Ressources

11.1 Équipe Recommandée

- **1 Tech Lead/Architecte** (full-time)
- **2 Développeurs Backend** Python/FastAPI
- **2 Développeurs Frontend** React/TypeScript
- **1 Développeur Mobile** React Native
- **1 Data Scientist** (algorithmes agricoles)
- **1 DevOps Engineer** (temps partiel)

11.2 Infrastructure Estimée (mensuel)

- **Serveurs cloud** : 500-1000€
- **Base de données managée** : 200-400€
- **CDN et storage** : 100-200€
- **Services tiers** (ERA5, monitoring) : 300-500€
- **Total** : ~1100-2100€/mois

11.3 Coûts de Développement

- **Phase 1-5** : ~6 mois de développement
- **Budget développement** : Variable selon équipe
- **Maintenance annuelle** : ~20% du coût initial

12. Risques et Mitigation

12.1 Risques Techniques

- **Complexité ERA5** : Prototypage précoce requis
- **Performance mobile** : Tests réguliers sur devices
- **Scalabilité** : Architecture cloud-native obligatoire

12.2 Risques Métier

- **Adoption utilisateur** : UX/UI iterations fréquentes
- **Exactitude conseils** : Validation avec experts agricoles
- **Compliance** : Veille réglementaire continue

13. Success Metrics

13.1 KPIs Produit

- **Utilisateurs actifs** : 1000+ riziculteurs première année
- **Rétention** : 70% à 3 mois
- **Satisfaction** : NPS > 50
- **Impact** : 15% amélioration décisions semis

13.2 KPIs Techniques

- **Uptime** : 99.9%
- **Performance** : < 200ms temps réponse
- **Couverture tests** : > 90%
- **Sécurité** : 0 incident critique

Ce cahier des charges constitue un document évolutif qui sera mis à jour selon les retours utilisateurs et les contraintes techniques découvertes pendant le développement.